SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 –Dataset Intégration Task

Course Code:	DSA0502	**Course Title:**	Query Processing for Data Science
CO Assessed:	CO3	**Assessment:**	Tool 3 - Dataset Intégration Task
Weightage:		**Total Marks:**	
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).			
Student Name:	Dharshini H	**Reg. No.:**	192324268
Section / Batch:	B.Tech AI and DS	**Date of Submission:**	16-08-2026
Faculty In-charge:	**Dr. B.Shyamala Gowri**	**Project Mode:**	Individual Submission

Code of Conduct

I Dharshini H, 192324268 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

Dataset Intégration Task

Question 1 – Customer Dataset Integration

Three customer datasets from **Online, Retail, and Support systems** contain inconsistent names, phone numbers, email addresses, dates, duplicate records, and missing values.

Task: Using Python/Pandas, clean and integrate the datasets by identifying invalid values, formatting data, removing duplicates, applying fuzzy matching and RegEx validation, standardizing fields, and saving the final master dataset in CSV and JSON formats. Create a reusable cleanup script and test it with a new dataset.

CO3 > at3_1.py > test_new_dataset

```python
import pandas as pd
import re
from rapidfuzz import fuzz
import os
# ------------------------------------------------------------
# CREATE OUTPUT FOLDER
# ------------------------------------------------------------
os.makedirs("output", exist_ok=True)
# ------------------------------------------------------------
# LOAD DATASET
# ------------------------------------------------------------
def load_dataset(file_path, source):
    df = pd.read_csv(file_path)
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_")
    )
    df["source"] = source
    return df
# ------------------------------------------------------------
# CLEAN NAME
# ------------------------------------------------------------
def clean_name(name):
    if pd.isna(name):
        return None
    name = str(name).strip()
```

CO3 > at3_1.py > test_new_dataset

```python
def clean_name(name):
    if pd.isna(name):
        return None
    name = str(name).strip()
    # Remove extra spaces
    name = re.sub(r"\s+", " ", name)
    # Keep only alphabets and spaces
    name = re.sub(r"[^A-Za-z\s]", "", name)
    # Title case
    name = name.title()
    return name

# ------------------------------------------------------------
# CLEAN PHONE
# ------------------------------------------------------------
def clean_phone(phone):
    if pd.isna(phone):
        return None
    phone = str(phone)
    # Remove spaces, +, -, brackets, etc.
    phone = re.sub(r"\D", "", phone)
    # Remove Indian country code
    if phone.startswith("91") and len(phone) == 12:
        phone = phone[2:]
    # Validate 10 digit number
    if len(phone) == 10:
        return phone
    return None
```

CO3 > at3_1.py > test_new_dataset

```python
# ------------------------------------------------------------
# VALIDATE EMAIL USING REGEX
# ------------------------------------------------------------
def clean_email(email):
    if pd.isna(email):
        return None
    email = str(email).strip().lower()
    pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
    if re.match(pattern, email):
        return email
    return None

# ------------------------------------------------------------
# CLEAN DATE
# ------------------------------------------------------------
def clean_date(date):
    if pd.isna(date):
        return None
    result = pd.to_datetime(
        date,
        errors="coerce",
        dayfirst=True
    )
    if pd.isna(result):
        return None
    return result.strftime("%Y-%m-%d")


# ------------------------------------------------------------
# CLEAN COMPLETE DATASET
# ------------------------------------------------------------
def clean_dataset(df):
    df["name"] = df["name"].apply(clean_name)
    df["phone"] = df["phone"].apply(clean_phone)
    df["email"] = df["email"].apply(clean_email)
    df["date"] = df["date"].apply(clean_date)
    # Convert empty values to NaN
    df = df.replace(r"^\s*$", pd.NA, regex=True)
    # Remove exact duplicate rows
    df = df.drop_duplicates()
    return df
# ------------------------------------------------------------
# FUZZY MATCHING
# ------------------------------------------------------------
def similarity(name1, name2):
    if pd.isna(name1) or pd.isna(name2):
        return 0
    return fuzz.ratio(
        str(name1).lower(),
        str(name2).lower()
    )
```

CO3 > at3_1.py > test_new_dataset

```python
# ------------------------------------------------------------
# FIND MASTER CUSTOMER ID
# ------------------------------------------------------------
def assign_master_ids(df):
    master_records = []
    master_ids = []
    next_id = 1
    for _, row in df.iterrows():
        name = row["name"]
        phone = row["phone"]
        email = row["email"]
        matched_id = None
        for record in master_records:
            name_score = similarity(
                name,
                record["name"]
            )
            phone_match = (
                phone is not None
                and record["phone"] is not None
                and phone == record["phone"]
            )
            email_match = (
                email is not None
                and record["email"] is not None
                and email == record["email"]
            )
```

```
            # Same phone/email OR highly similar name
            if (
                phone_match
                or email_match
                or name_score >= 85
            ):
                matched_id = record["master_id"]
                break
        if matched_id is None:
            matched_id = f"M{next_id:04d}"
            master_records.append({
                "master_id": matched_id,
                "name": name,
                "phone": phone,
                "email": email
            })
            next_id += 1
        master_ids.append(matched_id)
    df["MasterCustomerID"] = master_ids
    return df


# ------------------------------------------------------------
# INTEGRATE DATASETS
# ------------------------------------------------------------
def integrate_datasets():
    online = load_dataset(
        r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\online.csv",
        "Online"
    )
    retail = load_dataset(
        r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\retail.csv",
        "Retail"
    )
    support = load_dataset(
        r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\support.csv",
        "Support"
    )
    # Clean each dataset
    online = clean_dataset(online)
    retail = clean_dataset(retail)
    support = clean_dataset(support)
    # Combine datasets
    combined = pd.concat(
        [online, retail, support],
        ignore_index=True
    )
    return combined


"
# SAVE OUTPUT
# ------------------------------------------------------------
def save_output(df):
    csv_path = "output/master_customer.csv"
    json_path = "output/master_customer.json"
    df.to_csv(
        csv_path,
        index=False
    )
    df.to_json(
        json_path,
        orient="records",
        indent=4
    )
    print("\nOutput files created:")
    print(csv_path)
    print(json_path)
# ------------------------------------------------------------
# TEST NEW DATASET
# ------------------------------------------------------------
def test_new_dataset():
    print("\nTesting reusable script with new dataset...")
    test = load_dataset(
        r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\new_test_data.csv",
        "NewTest"
    )
    test = clean_dataset(test)
    test = assign_master_ids(test)
```

```python
# ------------------------------------------------------------
# TEST NEW DATASET
# ------------------------------------------------------------
def test_new_dataset():
    print("\nTesting reusable script with new dataset...")
    test = load_dataset(
        r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\new_test_data.csv",
        "NewTest"
    )
    test = clean_dataset(test)
    test = assign_master_ids(test)
    print("\nNew Dataset After Cleaning:")
    print(test.to_string(index=False))
```

```python
def main():
    print("======================================")
    print(" CUSTOMER DATA INTEGRATION SYSTEM")
    print("======================================")
    # Integrate three datasets
    df = integrate_datasets()
    print("\nTotal records after cleaning:")
    print(len(df))
    # Assign master customer IDs
    df = assign_master_ids(df)
    # Remove duplicate customer records
    df = df.drop_duplicates(
        subset=[
            "MasterCustomerID",
            "name",
            "phone",
            "email"
        ]
    )
    # Save final master dataset
    save_output(df)
    print("\nFinal Master Dataset:")
    print(df.to_string(index=False))
    # Test reusable script
    test_new_dataset()
# RUN PROGRAM
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Total records after cleaning:
12

Output files created:
output/master_customer.csv
output/master_customer.json

Final Master Dataset:
customerid         name      phone                  email       date source MasterCustomerID
      C001   John Smith 9876543210   john.smith@gmail.com 2026-10-01 Online            M0001
      C002 Priya Sharma 9876543211 priya.sharma@gmail.com 2026-01-10 Online            M0002
      C003  Rahul Kumar 9876543212        rahul@gmail.com 2026-02-15 Online            M0003
      C004    Anita Rao 9876543213        anita@gmail.com 2026-02-15 Online            M0004
      R102      Priya S 9876543211 priya.sharma@gmail.com 2026-10-01 Retail            M0002
      R104   Anitha Rao 9876543213        anita@gmail.com 2026-02-15 Retail            M0004

Testing reusable script with new dataset...
c:\Users\dhars\Documents\Query processing\lab_questions\CO3\at3_1.py:71: UserWarning: Parsing dates in %Y/%m/%d format wh
 dayfirst=True was specified. Pass `dayfirst=False` or specify a format to silence this warning.
  result = pd.to_datetime(

New Dataset After Cleaning:
customerid         name      phone                  email       date  source MasterCustomerID
      N001   John Smith 9876543210   john.smith@gmail.com 2026-08-16 NewTest            M0001
      N002   Anitha Rao 9876543213        anita@gmail.com 2026-08-16 NewTest            M0002
      N003 New Customer 9876543299  newcustomer@gmail.com 2026-08-16 NewTest            M0003
      N004 Priya Sharma 9876543211 priya.sharma@gmail.com 2026-08-16 NewTest            M0004
      N005  Wrong Email 9876543300                   NaN 2026-08-16 NewTest            M0005
PS C:\Users\dhars\Documents\Query processing\lab_questions>
```

Question 2 – Employee Dataset Integration

An organization has employee data stored in **HR, Payroll, and Attendance datasets**. The datasets contain different column names, inconsistent employee names, duplicate employees, invalid email IDs, different date formats, missing department values, and salary inconsistencies.

Task: Develop a Python program to integrate the three datasets. Apply data formatting, RegEx matching, duplicate detection, fuzzy matching, normalization/standardization, outlier detection, and missing-value handling. Save the cleaned integrated dataset and test the cleanup script using new employee records.

```python
import pandas as pd
import re
from rapidfuzz import fuzz
import os
# 1. LOAD DATA
def load_dataset(file_path, source):
    df = pd.read_csv(file_path)
    # Standardize column names
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_")
    )
    df["source"] = source
    return df

# 2. STANDARDIZE NAME
def clean_name(name):
    if pd.isna(name):
        return None
    name = str(name).strip()
    # Remove extra spaces
    name = re.sub(r"\s+", " ", name)
    # Remove unwanted characters
    name = re.sub(r"[^A-Za-z\s]", "", name)
    return name.title()

# 3. EMAIL VALIDATION USING REGEX
def clean_email(email):
    if pd.isna(email):
        return None
    email = str(email).strip().lower()
    pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
    if re.match(pattern, email):
        return email
    return None

# 4. STANDARDIZE DEPARTMENT
def clean_department(department):
    if pd.isna(department):
        return None
    department = str(department).strip().lower()
    department = re.sub(r"\s+", " ", department)
    department_mapping = {
        "it": "Information Technology",
        "information technology": "Information Technology",
        "hr": "Human Resources",
        "human resources": "Human Resources",
        "finance": "Finance",
        "marketing": "Marketing"
    }
    return department_mapping.get(
        department,
        department.title()
```

```python
# 5. STANDARDIZE DATE
def clean_date(date):
    if pd.isna(date):
        return None
    date = str(date).strip()
    # YYYY-MM-DD / YYYY/MM/DD
    if re.match(r"^\d{4}[-/]\d{2}[-/]\d{2}$", date):
        result = pd.to_datetime(
            date,
            errors="coerce",
            yearfirst=True
        )
    # DD-MM-YYYY / DD/MM/YYYY
    else:
        result = pd.to_datetime(
            date,
            errors="coerce",
            dayfirst=True
        )
    if pd.isna(result):
        return None
    return result.strftime("%Y-%m-%d")
# 6. CLEAN SALARY
def clean_salary(salary):
    if pd.isna(salary):
        return None
    try:
        salary = float(
            str(salary)
            .replace(",", "")
            .replace("₹", "")
        )
        return salary
    except:
        return None
```

```python
# 7. CLEAN DATASET
def clean_dataset(df):
    # Rename different columns
    rename_columns = {
        "empid": "employee_id","employee_id": "employee_id",
        "employee_name": "name","name": "name",
        "dept": "department","department": "department",
        "joiningdate": "joining_date",
        "joining_date": "joining_date",
        "paymentdate": "payment_date",
        "payment_date": "payment_date",
        "attendancedate": "attendance_date",
        "attendance_date": "attendance_date"
    }
    df = df.rename(columns=rename_columns)
    # Clean fields
    if "name" in df.columns:
        df["name"] = df["name"].apply(clean_name)
    if "email" in df.columns:
        df["email"] = df["email"].apply(clean_email)
    if "department" in df.columns:
        df["department"] = df["department"].apply(
            clean_department
        )
    if "joining_date" in df.columns:
        df["joining_date"] = df["joining_date"].apply(
            clean_date
        )
    if "payment_date" in df.columns:
        df["payment_date"] = df["payment_date"].apply(
            clean_date
        )
    if "attendance_date" in df.columns:
        df["attendance_date"] = df["attendance_date"].apply(
            clean_date
        )
    if "salary" in df.columns:
        df["salary"] = df["salary"].apply(
            clean_salary
        )
    df = df.drop_duplicates()
    return df
# 8. FUZZY MATCHING
def name_similarity(name1, name2):
    if pd.isna(name1) or pd.isna(name2):
        return 0
    return fuzz.ratio(
        str(name1).lower(),
        str(name2).lower()
    )

# 9. DETECT SALARY OUTLIERS
def detect_salary_outliers(df):
    if "salary" not in df.columns:
        return df
    salary = df["salary"]
    Q1 = salary.quantile(0.25)
    Q3 = salary.quantile(0.75)
    IQR = Q3 - Q1
    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR
    df["salary_outlier"] = (
        (salary < lower_limit) |
        (salary > upper_limit)
    )
    return df
```

```python
# 10. ASSIGN MASTER EMPLOYEE ID
def assign_master_ids(df):
    master_records = []
    master_ids = []
    next_id = 1
    for _, row in df.iterrows():
        name = row.get("name")
        email = row.get("email")
        employee_id = row.get("employee_id")
        matched_id = None
        for record in master_records:
            name_score = name_similarity(
                name,
                record["name"]
            )
            email_match = (
                email is not None
                and record["email"] is not None
                and email == record["email"]
            )
            employee_match = (
                employee_id is not None
                and record["employee_id"] is not None
                and employee_id == record["employee_id"]
            )
            if (
                email_match
                or employee_match
                or name_score >= 85
            ):
                matched_id = record["master_id"]
                break
        if matched_id is None:
            matched_id = f"ME{next_id:04d}"
            master_records.append({
                "master_id": matched_id,"name": name,
                "email": email,
                "employee_id": employee_id
            })
            next_id += 1
        master_ids.append(matched_id)
    df["MasterEmployeeID"] = master_ids
    return df
# 11. INTEGRATE HR, PAYROLL, ATTENDANCE
def integrate_datasets():
    base_path = r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3"
    hr = load_dataset(
        os.path.join(base_path, r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\hr.csv"),
        "HR"
    )
    payroll = load_dataset(
        os.path.join(base_path, r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\payroll.csv"),
        "Payroll"
    )
    attendance = load_dataset(
        os.path.join(base_path, r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\attendance.csv"),
        "Attendance"
    )
    hr = clean_dataset(hr)
    payroll = clean_dataset(payroll)
    attendance = clean_dataset(attendance)
    combined = pd.concat(
        [
            hr,
            payroll,
            attendance
        ],
        ignore_index=True,
        sort=False
    )
    return combined
# 12. MISSING VALUE HANDLING
def handle_missing_values(df):
    # Department
    if "department" in df.columns:
        df["department"] = df["department"].fillna(
            "Unknown"
        )
    # Email
    if "email" in df.columns:
        df["email"] = df["email"].fillna(
            "Not Available"
        )
    # Salary
    if "salary" in df.columns:
        df["salary"] = df["salary"].fillna(
            df["salary"].median()
        )
    return df

# 13. SAVE DATA
def save_output(df):
    output_folder = r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\output"
    os.makedirs(
        output_folder,
        exist_ok=True
    )
    csv_path = os.path.join(
        output_folder,
        "master_employee.csv"
    )
    json_path = os.path.join(
        output_folder,
        "master_employee.json"
    )
    df.to_csv(
        csv_path,
        index=False
    )
    df.to_json(
        json_path,
        orient="records",
        indent=4
    )
```

```python
# 14. TEST NEW DATASET
def test_new_dataset():
    print("\n======================================")
    print(" TESTING NEW EMPLOYEE DATA")
    print("======================================")
    path = (
        r"C:\Users\dhars\Documents\Query processing\lab_questions\CO3\new_employee.csv"
    )
    test = load_dataset(
        path,
        "NewRecords"
    )
    test = clean_dataset(test)
    test = assign_master_ids(test)
    test = detect_salary_outliers(test)
    test = handle_missing_values(test)
    print("\nCleaned New Employee Records:")
    print(
        test.to_string(index=False)
    )
# 15. MAIN
def main():
    print("======================================")
    print(" EMPLOYEE DATA INTEGRATION SYSTEM")
    print("======================================")
    df = integrate_datasets()
    print(
        "\nTotal records after cleaning:",
        len(df)
    )
    df = assign_master_ids(df)
    df = detect_salary_outliers(df)
    df = handle_missing_values(df)
    df = df.drop_duplicates(
        subset=[
            "MasterEmployeeID",
            "name",
            "email"
        ]
    )
    save_output(df)
    print("\nFinal Integrated Dataset:")
    print(
        df.to_string(index=False)
    )
    test_new_dataset()
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Total records after cleaning: 30

Files created:
C:\Users\dhars\Documents\Query processing\lab_questions\CO3\output\master_employee.csv
C:\Users\dhars\Documents\Query processing\lab_questions\CO3\output\master_employee.json

Final Integrated Dataset:
employeeid employeename                email            department joining_date   salary     source employee_id        name payment_date attendance_date  status MasterEmployeeID  salary_outlier
      E001   Arun Kumar   arun.kumar@gmail.com Information Technology   2024-01-10  55000.0         HR         NaN         NaN             NaN     NaN            ME0001           False
      E002 Priya Sharma priya.sharma@gmail.com         Human Resources   2024-02-15  60000.0         HR         NaN         NaN             NaN     NaN            ME0002           False
      E003  Rahul Kumar  rahul.kumar@gmail.com                Finance   2024-03-20  58000.0         HR         NaN         NaN             NaN     NaN            ME0003           False
      E004    Anita Rao    anita.rao@gmail.com Information Technology   2024-04-05  62000.0         HR         NaN         NaN             NaN     NaN            ME0004           False
      E005  Sneha Patel  sneha.patel@gmail.com              Marketing   2024-05-10  52000.0         HR         NaN         NaN             NaN     NaN            ME0005           False
      E006  Vijay Kumar  vijay.kumar@gmail.com                Finance   2024-06-15  59000.0         HR         NaN         NaN             NaN     NaN            ME0006           False
      E007   Meena Devi   meena.devi@gmail.com        Human Resources   2024-06-20  61000.0         HR         NaN         NaN             NaN     NaN            ME0007           False
      E008  Krishna Raj  krishna.raj@gmail.com Information Technology   2024-07-25  65000.0         HR         NaN         NaN             NaN     NaN            ME0008           False
      E009   Neha Singh   neha.singh@gmail.com              Marketing   2024-08-10  54000.0         HR         NaN         NaN             NaN     NaN            ME0009           False
      E010  Ravi Sharma  ravi.sharma@gmail.com                Finance   2024-08-15  57000.0         HR         NaN         NaN             NaN     NaN            ME0010           False
       NaN          NaN   arun.kumar@gmail.com Information Technology          NaN  55000.0    Payroll        E001   Arun Kumar      2026-01-31             NaN     NaN            ME0001           False
       NaN          NaN priya.sharma@gmail.com         Human Resources          NaN  60000.0    Payroll        E002      Priya S      2026-01-31             NaN     NaN            ME0002           False
       NaN          NaN  rahul.kumar@gmail.com                Finance          NaN  58000.0    Payroll        E003  Rahul Kumar      2026-01-31             NaN     NaN            ME0003           False
       NaN          NaN    anita.rao@gmail.com Information Technology          NaN  62000.0    Payroll        E004    Anitha Rao     2026-01-31             NaN     NaN            ME0004           False
       NaN          NaN  sneha.patel@gmail.com              Marketing          NaN  52000.0    Payroll        E005  Sneha Patel      2026-01-31             NaN     NaN            ME0005           False
       NaN          NaN  vijay.kumar@gmail.com                Finance          NaN  59000.0    Payroll        E006  Vijay Kumar      2026-01-31             NaN     NaN            ME0006           False
       NaN          NaN   meena.devi@gmail.com        Human Resources          NaN  61000.0    Payroll        E007   Meena Devi      2026-01-31             NaN     NaN            ME0007           False
       NaN          NaN  krishna.raj@gmail.com Information Technology          NaN 150000.0    Payroll        E008  Krishna Raj      2026-01-31             NaN     NaN            ME0008            True
       NaN          NaN          Not Available              Marketing          NaN  54000.0    Payroll        E009   Neha Singh      2026-01-31             NaN     NaN            ME0011           False
       NaN          NaN  ravi.sharma@gmail.com                Finance          NaN  57000.0    Payroll        E010  Ravi Sharma      2026-01-31             NaN     NaN            ME0010           False
       NaN          NaN priya.sharma@gmail.com                Unknown          NaN  58500.0 Attendance        E002 Priya Sharma             NaN      2026-08-01 Present            ME0002           False
       NaN          NaN    anita.rao@gmail.com Information Technology          NaN  58500.0 Attendance        E004    Anita Rao             NaN      2026-08-01 Present            ME0004           False
       NaN          NaN   neha.singh@gmail.com                Unknown          NaN  58500.0 Attendance        E009    Neha Singh            NaN      2026-08-01  Absent            ME0009           False

======================================
 TESTING NEW EMPLOYEE DATA
======================================

Cleaned New Employee Records:
employeeid employeename               email            department joining_date   salary     source MasterEmployeeID  salary_outlier
      N001  Arun  Kumar  arun.kumar@gmail.com Information Technology   2026-08-15  55000.0 NewRecords           ME0001           False
      N002   Anitha Rao   anita.rao@gmail.com Information Technology   2026-08-16  62000.0 NewRecords           ME0002           False
      N003 New Employee newemployee@gmail.com Information Technology   2026-08-17  58000.0 NewRecords           ME0003           False
      N004      Priya S         Not Available               Unknown   2026-08-18 200000.0 NewRecords           ME0004            True
PS C:\Users\dhars\Documents\Query processing\lab_questions>
```